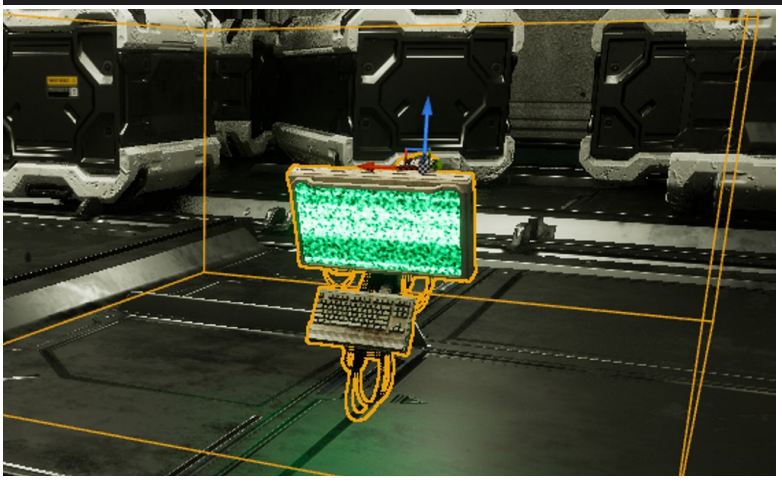
**Hart, Noah - u1537288 - Actively Contributing**

This Sprint, we held a meeting to discuss a refined, scoped-down vision of our final goals. This included refactoring and changing certain elements of level design for both the station and train cars based on player feedback and workloads. We have decided to make stations more linear with multiple paths to explore in a short time limit, risking reward with death if you take too long to loot. On top of the loot you unlock along each path, you can unlock certain doors around the station to continue to explore, with the completion of one main path unlocking the other. This scaled-down approach allows us to dedicate more time to other areas of our project.

My main task was to refactor the encounter manager and make it more modular, allowing Peng, our new traincar level designer, to implement and test layouts and encounters without my help. I also put together info in our tech doc describing certain processes for the people using my systems, like how to set up a level and get aligned properly. The main overview for this is that I made it so we have the manage spawn to pools of levels at random for a total of two encounters per trainphase. For level design, we brought on Brian to help whitebox our station level so that I can focus on the more important engineering task. Part of making the encounters more moduallar I created a new actor that allows you to select the encounter manually based on the layout you put it in.

```
#include "CoreMinimal.h"
#include "EncounterType.generated.h"

UENUM(BlueprintType)
enum class EEncounterType : uint8
{
    SpawnWave     UMETA(DisplayName = "Spawn Wave"),
    CaptureZone   UMETA(DisplayName = "Capture Zone"),
    //DefendTower   UMETA(DisplayName = "Defend Tower")
};
```



```cpp
// ============================================================
// ================== ENCOUNTER START (TERMINAL) ================
// ============================================================
void AEncounterManager::StartEncounterByType(EEncounterType Type, ULevel* TargetLevel)
{
    if (!TargetLevel)
    {
        UE_LOG(LogTemp, Warning, TEXT("StartEncounterByType: TargetLevel is null."));
        return;
    }

    // Find the streamed level instance that owns this ULevel (so we can SetOwningLevel for encounters)
    ULevelStreamingDynamic* OwningStream = GetStreamingForLoadedLevel(TargetLevel);
    if (!OwningStream)
    {
        UE_LOG(LogTemp, Warning, TEXT("StartEncounterByType: Could not find streaming instance for TargetLevel."));
    }

    TSubclassOf<AActor> EncounterClass = nullptr;

    switch (Type)
    {
    case EEncounterType::SpawnWave:
        EncounterClass = SpawnWaveEncounterClass;
        break;

    case EEncounterType::CaptureZone:
        EncounterClass = CaptureZoneEncounterClass;
        break;

    //case EEncounterType::DefendTower:
    //    EncounterClass = DefendTowerEncounterClass;
    //    break;

    default:
        break;
    }

    if (!EncounterClass)
    {
        UE_LOG(LogTemp, Warning, TEXT("StartEncounterByType: EncounterClass not set for type."));
        return;
    }
```

